**C:\hpc-23-feb-2026\.\22\eg169.c**

```
#include<stdio.h>
#include<stdatomic.h>
int main()
{
 _Atomic int x=10;
 int y;
 y=atomic_load(&x);
 printf("%d\n",y);
 atomic_store(&x,55);
 y=atomic_load(&x);
 printf("%d\n",y);
 atomic_fetch_add(&x,10);
 y=atomic_load(&x);
 printf("%d\n",y);
 atomic_fetch_sub(&x,5);
 y=atomic_load(&x);
 printf("%d\n",y);
 return 0;
}
```

Thinking Machines Classroom Codes

**C:\hpc-23-feb-2026\.\22\eg170.cpp**

```
#include<iostream>
#include<thread>
void do_something()
{
 // some code
}
int main()
{
 thread t(do_something);
 // some code
 //
 // some code, may be return statement
 //
 //
 // some code, may be a return statement
 //
 //
 // some code, may be a function call that may raise exception
 //
 //
 // some code
 t.join();
return 0;
}
```

**C:\hpc-23-feb-2026\.\22\eg171.cpp**

```
#include<iostream>
#include<thread>
void do_something()
{
 // some code
}
int main()
{
 try
 {
 thread t(do_something);
 some_class obj(t); // parameterized constructor with (thread &t):t(t){}
   // some class property -- thread &t;
   // destructor with t.join();
    // some_class should/will be called thread_guard
 // some code
 //
 // some code, may be return statement
 //
 //
 // some code, may be a return statement
 //
 //
 // some code, may be a function call that may raise exception
 //
 //
 // some code
 }catch(...)
 {
 }
return 0;
}
```

**C:\hpc-23-feb-2026\.\22\eg172.cpp**

```
#include<iostream>
#include<vector>
using namespace std;
int main()
{
 vector<int> v={10,20,30,40};
 auto iter=v.begin();
 cout<<*iter<<endl;
 auto k=iter++;
 cout<<*k<<endl;
 cout<<*iter<<endl;
 return 0;
}
```

Thinking Machines Classroom Codes

**C:\hpc-23-feb-2026\.\22\eg173.cpp**

```
#include<iostream>
#include<vector>
using namespace std;
int main()
{
 vector<int> v={10,20,30,40};
 auto iter=v.begin();
 cout<<*iter<<endl;
 auto k=++iter;
 cout<<*k<<endl;
 cout<<*iter<<endl;
 return 0;
}
```

**C:\hpc-23-feb-2026\.\22\eg174.cpp**

```
#include<iostream>
using namespace std;
class Collection
{
 int x[4];
 public:
 Collection()
 {
  x[0]=10;
  x[1]=20;
  x[2]=30;
  x[3]=40;
 }
 class iterator
 {
  public:
   int *ptr;
   int index;
   iterator(int *ptr,int index):ptr(ptr),index(index){}
   int operator!=(const iterator &other)
   {
    return this->index!=other.index;
   }
   // ideally other relational operators should be overloaded
   int operator*()
   {
    return this->ptr[this->index];
   }
   iterator operator++() // for prefix
   {
    ++(this->index);
    return *this;
   }
   iterator operator++(int) // for postfix
   {
    iterator k=*this;
    this->index++;
    return k;
   }
};
iterator begin()
{
 iterator t(x,0);
 return t;
}
iterator end()
{
 iterator t(NULL,4);
```

```
 return t;
 }
};
int main()
{
 Collection c;
 for(Collection::iterator iter=c.begin();iter!=c.end();++iter)
 {
  cout<<*iter<<endl;
 }
 cout<<"----------------------------"<<endl;
 for(auto iter=c.begin();iter!=c.end();++iter)
 {
  cout<<*iter<<endl;
 }
 cout<<"----------------------------"<<endl;
 for(auto x:c)
 {
  cout<<x<<endl;
 }
 return 0;
}
```

**C:\hpc-23-feb-2026\.\22\eg175.cpp**

```
#include<iostream>
#include<vector>
#include<chrono>
using namespace std;
int main()
{
vector<int> v(100000000,10);
cout<<v[3]<<endl;
cout<<v.size()<<endl;
auto start=chrono::high_resolution_clock::now();
for(auto iter=v.begin();iter!=v.end();++iter)
{
 *iter=(*iter)*(*iter);
}
auto end=chrono::high_resolution_clock::now();
auto duration=chrono::duration_cast<chrono::milliseconds>(end-start);
cout<<duration.count()<<" milliseconds"<<endl;
cout<<v[3]<<endl;
return 0;
}
```

**C:\hpc-23-feb-2026\.\22\eg176.cpp**

```
#include<iostream>
#include<vector>
#include<chrono>
using namespace std;
int main()
{
vector<int> v(100000000,10);
cout<<v[3]<<endl;
cout<<v.size()<<endl;
auto start=chrono::high_resolution_clock::now();
auto e=v.end();
for(auto iter=v.begin();iter!=e;++iter)
{
 *iter=(*iter)*(*iter);
}
auto end=chrono::high_resolution_clock::now();
auto duration=chrono::duration_cast<chrono::milliseconds>(end-start);
cout<<duration.count()<<" milliseconds"<<endl;
cout<<v[3]<<endl;
return 0;
}
```

**C:\hpc-23-feb-2026\.\22\eg177.cpp**

```
#include<iostream>
#include<vector>
#include<chrono>
using namespace std;
int main()
{
vector<int> v(100000000,10);
cout<<v[3]<<endl;
cout<<v.size()<<endl;
auto start=chrono::high_resolution_clock::now();
auto e=v.end();
for(auto &k:v)
{
 k=k*k;
}
auto end=chrono::high_resolution_clock::now();
auto duration=chrono::duration_cast<chrono::milliseconds>(end-start);
cout<<duration.count()<<" milliseconds"<<endl;
cout<<v[3]<<endl;
return 0;
}
```

**C:\hpc-23-feb-2026\.\22\eg178.cpp**

```
#include<iostream>
#include<vector>
#include<chrono>
using namespace std;
int main()
{
vector<int> v(100000000,10);
cout<<v[3]<<endl;
cout<<v.size()<<endl;
auto start=chrono::high_resolution_clock::now();
auto sz=v.size();
int *p=v.data();
for(int i=0;i<sz;++i)
{
 int x=*p;
 x*=x;
 *p=x;
}
auto end=chrono::high_resolution_clock::now();
auto duration=chrono::duration_cast<chrono::milliseconds>(end-start);
cout<<duration.count()<<" milliseconds"<<endl;
cout<<v[3]<<endl;
return 0;
}
```

Thinking Machines Classroom Codes